In [37]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
data = pd.read_csv("C:/Users/Lenovo/Downloads/advertising.csv")
print("First five row \n",data.head())
print("Last five rows\n ",data.tail())
print("Shape of the dataset: ",data.shape)
print(data.dtypes)
print("Missing values :\n", data.isnull().sum())
print("Duplicate values :\n", data.duplicated().sum())
print(data.describe())
x=data[["TV","Radio","Newspaper"]]
y=data["Sales"]
print(x.head())
print(y.head())
sns.pairplot(data[["TV","Radio","Newspaper"]])
plt.show()
plt.figure(figsize=(8,6))
correlation_matrix=data[["TV", "Radio", "Newspaper", "Sales"]].corr()
sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5)

plt.title("Correlation Heatmap")
plt.show()
sales_correlation = correlation_matrix["Sales"].drop("Sales")

print("Correlation with Sales:")
print(sales_correlation)

strongest_feature = sales_correlation.abs().idxmax()

print("\nFeature most strongly correlated with Sales:", strongest_feature)
X_train, X_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.20,
    random_state=42
)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])

linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

y_pred_linear = linear_model.predict(X_test)
```

```
print("Linear Regression Predictions:")
print(y_pred_linear[:10])
polynomial_model = Pipeline([
    ("poly", PolynomialFeatures(degree=2, include_bias=False)),
    ("linear", LinearRegression())
])
polynomial_model.fit(X_train, y_train)
y_pred_poly = polynomial_model.predict(X_test)
print("Polynomial Regression Predictions:")
print(y_pred_poly[:10])
svr_model = Pipeline([
    ("scaler", StandardScaler()),
    ("svr", SVR(kernel="rbf"))
])

# Train model
svr_model.fit(X_train, y_train)

# Predict
y_pred_svr = svr_model.predict(X_test)

print("SVR Predictions:")
print(y_pred_svr[:10])
def evaluate_model(model_name, y_actual, y_pred):
    mae = mean_absolute_error(y_actual, y_pred)
    mse = mean_squared_error(y_actual, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_actual, y_pred)

    return {
        "Model": model_name,
        "MAE": mae,
        "MSE": mse,
        "RMSE": rmse,
        "R2 Score": r2
    }


results = []

results.append(
    evaluate_model("Linear Regression", y_test, y_pred_linear)
)

results.append(
    evaluate_model("Polynomial Regression", y_test, y_pred_poly)
)

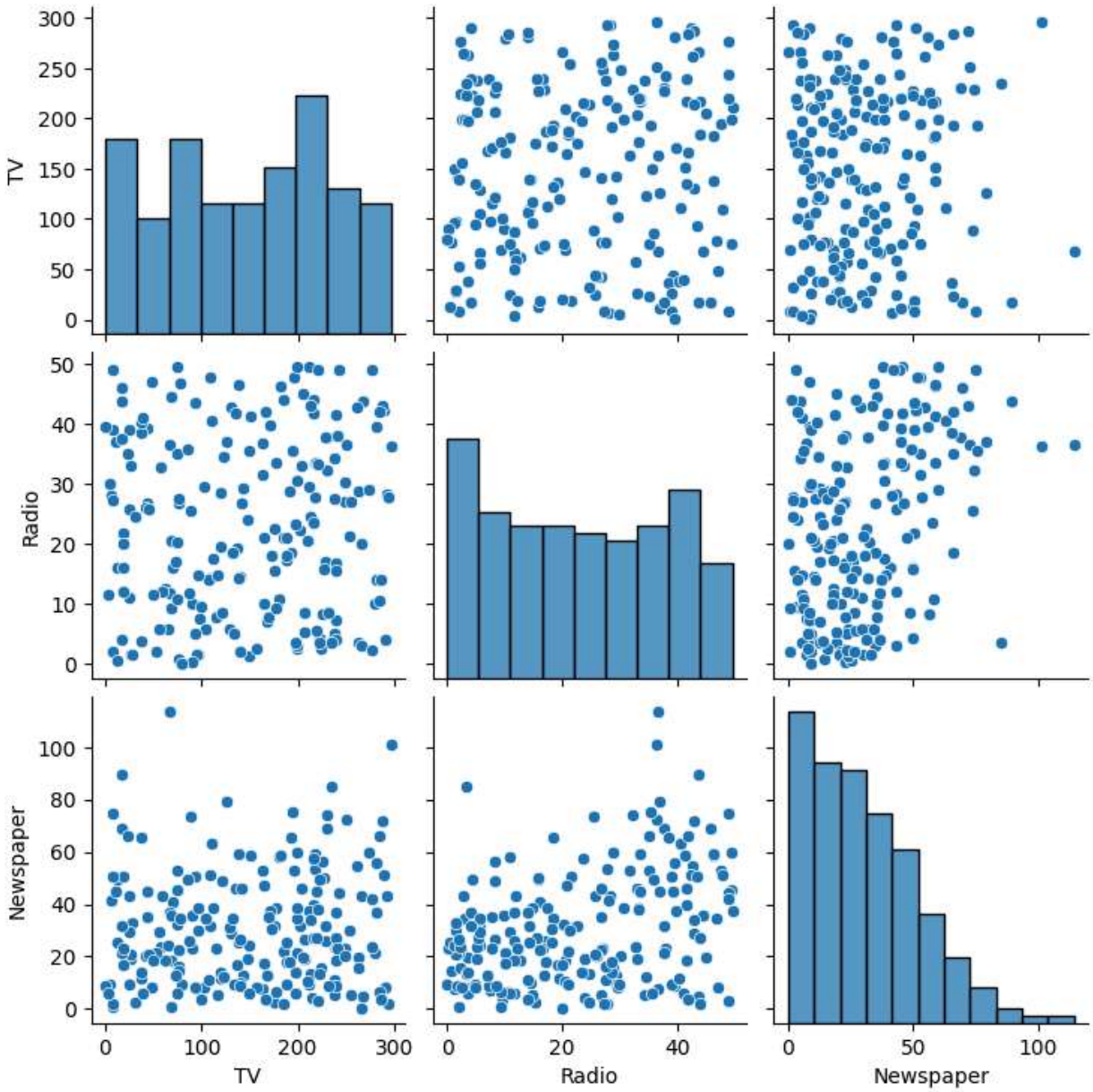
results.append(
    evaluate_model("SVR", y_test, y_pred_svr)
)

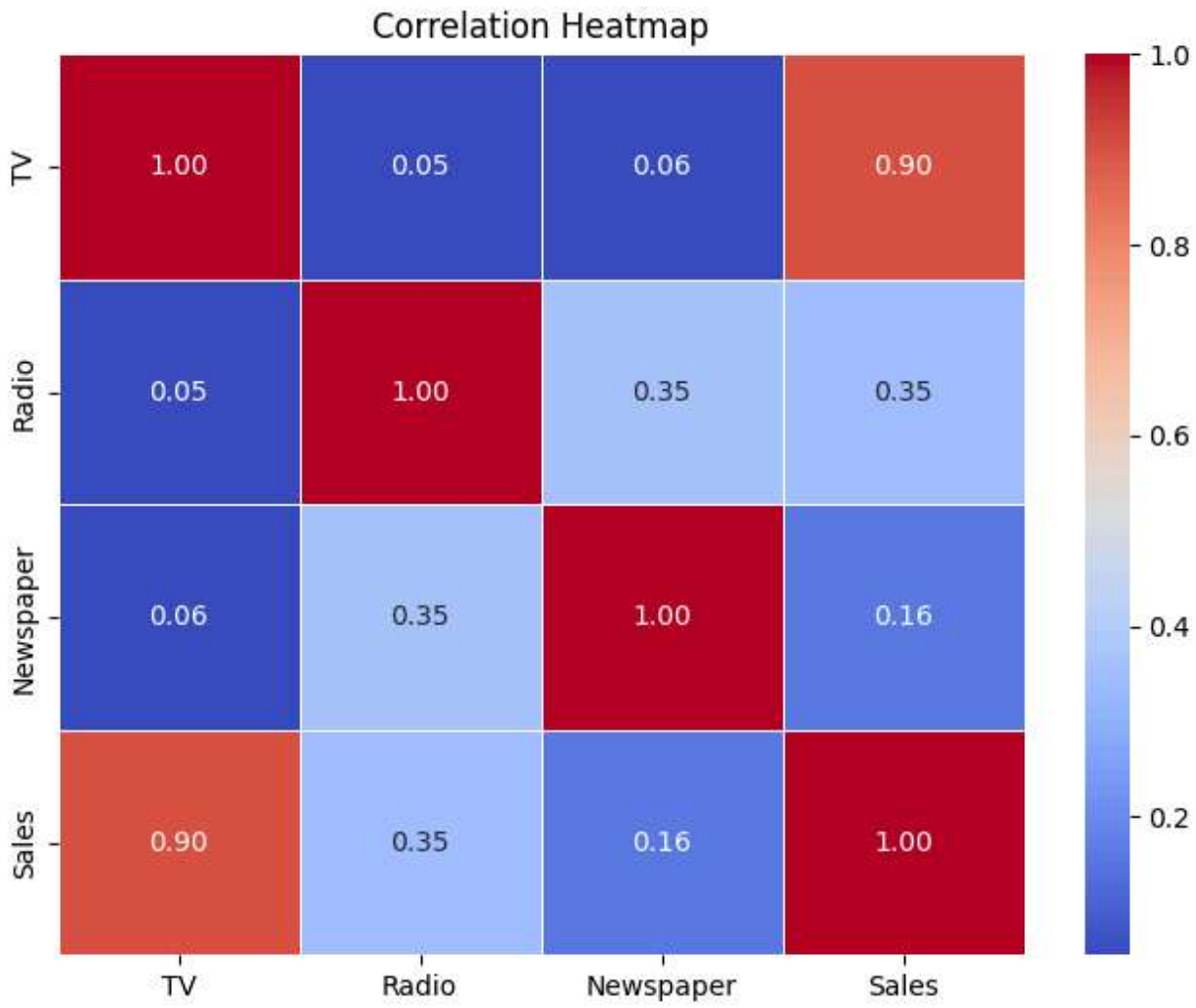
results_df = pd.DataFrame(results)

print("Model Performance:")
display(results_df.round(4))
best_model_rmse = results_df.loc[
    results_df["RMSE"].idxmin()
]
```

```
print("Best-performing model based on RMSE:")
print(best_model_rmse)
```

```
First five row
       TV  Radio  Newspaper  Sales
0  230.1   37.8       69.2   22.1
1   44.5   39.3       45.1   10.4
2   17.2   45.9       69.3   12.0
3  151.5   41.3       58.5   16.5
4  180.8   10.8       58.4   17.9
Last five rows
          TV  Radio  Newspaper  Sales
195   38.2    3.7       13.8    7.6
196   94.2    4.9        8.1   14.0
197  177.0    9.3        6.4   14.8
198  283.6   42.0       66.2   25.5
199  232.1    8.6        8.7   18.4
Shape of the dataset:  (200, 4)
TV           float64
Radio        float64
Newspaper    float64
Sales        float64
dtype: object
Missing values :
 TV           0
Radio        0
Newspaper    0
Sales        0
dtype: int64
Duplicate values :
 0
                TV       Radio   Newspaper       Sales
count  200.000000  200.000000  200.000000  200.000000
mean   147.042500   23.264000   30.554000   15.130500
std     85.854236   14.846809   21.778621    5.283892
min      0.700000    0.000000    0.300000    1.600000
25%     74.375000    9.975000   12.750000   11.000000
50%    149.750000   22.900000   25.750000   16.000000
75%    218.825000   36.525000   45.100000   19.050000
max    296.400000   49.600000  114.000000   27.000000
      TV  Radio  Newspaper
0  230.1   37.8       69.2
1   44.5   39.3       45.1
2   17.2   45.9       69.3
3  151.5   41.3       58.5
4  180.8   10.8       58.4
0    22.1
1    10.4
2    12.0
3    16.5
4    17.9
Name: Sales, dtype: float64
```


TV
Radio
Newspaper
0
50
100
150
200
250
300
0
10
20
30
40
50
0
20
40
60
80
100
0
100
200
300
TV
0
20
40
Radio
0
50
100
Newspaper




```
Correlation with Sales:
TV           0.901208
Radio        0.349631
Newspaper    0.157960
Name: Sales, dtype: float64

Feature most strongly correlated with Sales: TV
Training samples: 160
Testing samples: 40
Linear Regression Predictions:
[17.0347724  20.40974033 23.72398873  9.27278518 21.68271879 12.56940161
 21.08119452  8.69035045 17.23701254 16.66657475]
Polynomial Regression Predictions:
[17.562438   21.96285664 22.07580481  7.34916293 23.23684435 12.37109504
 22.12872506  8.76904874 17.01492635 16.91480608]
SVR Predictions:
[17.27716452 21.14815556 20.54171345 10.42918914 20.17484466 12.47291227
 21.43267317  9.09307441 17.33861046 16.54760086]
Model Performance:
```

	Model	MAE	MSE	RMSE	R2 Score
0	Linear Regression	1.2748	2.9078	1.7052	0.9059
1	Polynomial Regression	0.9034	1.4425	1.2011	0.9533
2	SVR	1.5554	4.4210	2.1026	0.8569

```
Best-performing model based on RMSE:
Model       Polynomial Regression
MAE                      0.903371
MSE                      1.442543
RMSE                     1.201059
R2 Score                 0.953317
Name: 1, dtype: object
```

In [ ]: